Введение

Цель работы - реализовать алгоритм сортировки TIMSORT на языке Python, сгенерировать 100 наборов входных данных, измерить время работы и количество шагов, а затем сравнить практические результаты с теоретической сложностью.

Измерения выполнялись только для самой сортировки. Чтение данных из файла, генерация входных массивов и построение графиков в измеряемое время не включались.

1. Название алгоритма и краткая историческая справка

TIMSORT - гибридный устойчивый алгоритм сортировки, сочетающий идеи сортировки слиянием и сортировки вставками. Алгоритм был предложен Тимом Питерсом для Python. Его важная особенность состоит в том, что он умеет использовать уже имеющийся порядок в данных, поэтому хорошо работает не только на случайных массивах, но и на почти отсортированных последовательностях.

2. Описание принципа работы и особенностей TIMSORT

Основные идеи TIMSORT можно описать так:

- массив просматривается слева направо и в нём ищутся естественные серии - уже отсортированные подпоследовательности;
- если серия оказалась убывающей, она разворачивается, чтобы получить возрастающий run;
- слишком короткие серии дополняются и сортируются вставками до минимального размера min_run;
- после этого серии постепенно сливаются между собой, пока не получится один отсортированный массив.

Преимущества алгоритма: устойчивость, хорошая работа на реальных данных и адаптивность. Недостаток учебной реализации - использование временных списков при слиянии, что требует дополнительной памяти.

3. Описание программной реализации

Проект состоит из трёх основных Python-файлов: timsort.py, data_generator.py и benchmark.py.

Файл timsort.py содержит сам алгоритм сортировки и функции get_min_run, insertion_sort, merge, find_run и timsort. Файл data_generator.py создаёт 100 текстовых файлов со случайными массивами. Файл benchmark.py читает каждый файл, запускает сортировку, измеряет время работы и количество шагов, сохраняет таблицу результатов и строит графики.

4. Генерация входных данных

Было сгенерировано 100 наборов случайных входных данных. Размеры массивов изменяются от 100 до 10 000 элементов.

Каждый файл содержит один массив целых чиселе.

5. Способы измерения работы алгоритма

По условию работы были реализованы два способа измерения:

- по времени выполнения - функция measure_time запускает сортировку несколько раз и берёт медианное значение времени в миллисекундах;
- по количеству шагов функция measure_steps запускает сортировку в режиме подсчёта и получает число сравнений, сдвигов и записей.

6. Таблицы результатов

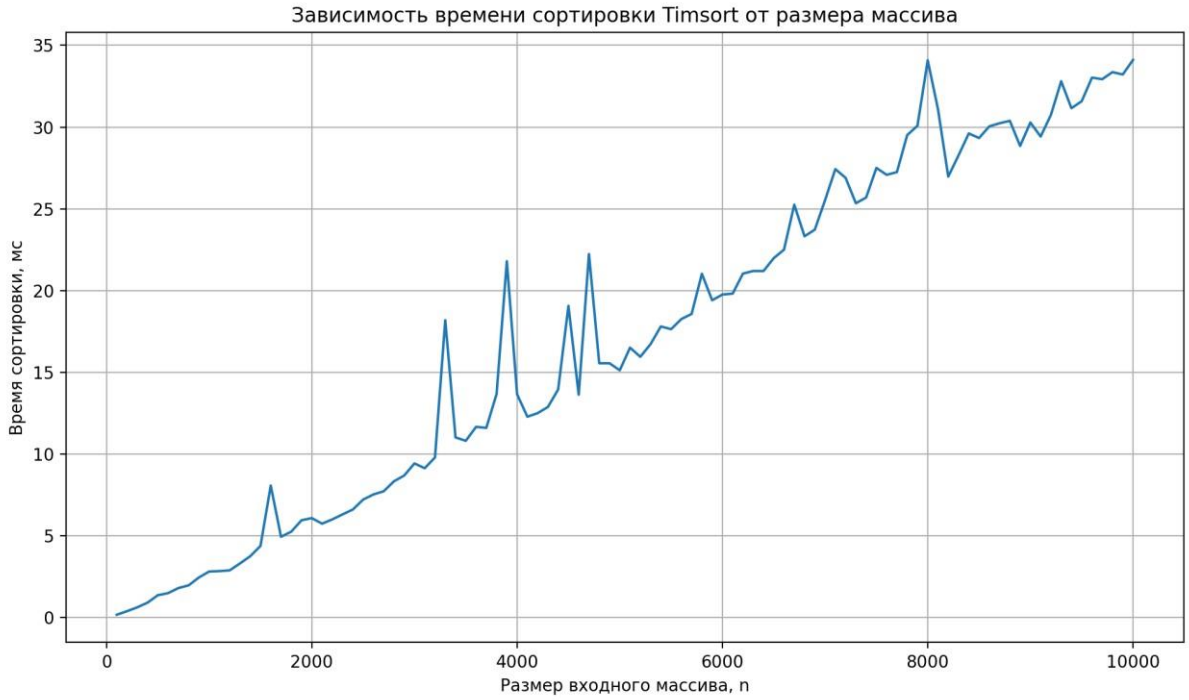
Сначала приведём краткую сводку по всем 100 экспериментам.

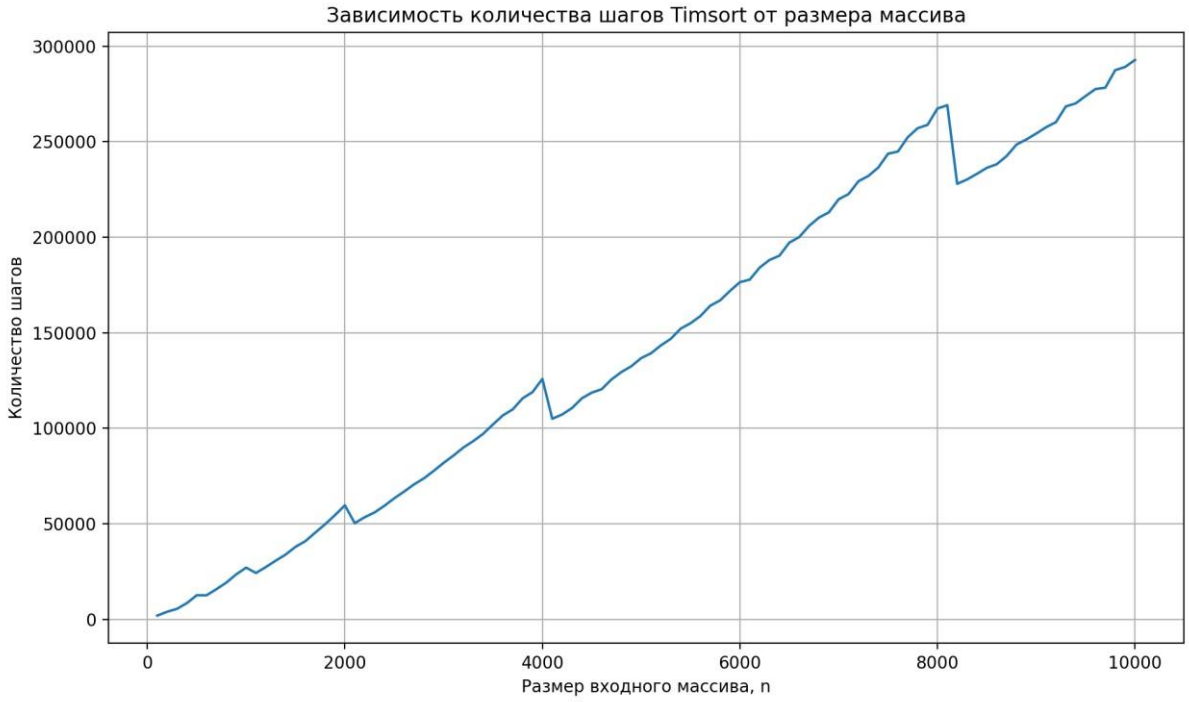
Показатель	Значение
Количество наборов данных	100
Минимальный размер массива	100
Максимальный размер массива	10000
Среднее время сортировки, мс	17.25
Минимальное время сортировки, мс	0.181232
Максимальное время сортировки, мс	34.106141
Среднее количество шагов	146256
Минимальное количество шагов	1936
Максимальное количество шагов	292719
Коэффициент корреляции size/time	0.9839
Коэффициент корреляции size/steps	0.9926

№ набора	Размер n	Время, мс	Шаги
1	100	0.181232	1936
10	1000	2.824304	27038
20	2000	6.089559	59675
30	3000	9.433453	81882
40	4000	13.657633	125814
50	5000	15.12835	136695
60	6000	19.760037	176445
70	7000	25.531605	219830
80	8000	34.08404	267312
90	9000	30.278229	254224
100	10000	34.106141	292719

7. Графики

Ниже приведены графики зависимости времени выполнения и количества шагов от размера входного массива.





Оценка сложности алгоритма и сравнение теории с практикой

Теоретически TIMSORT относится к адаптивным устойчивым алгоритмам. Для него обычно указывают:

- лучший случай - O(n), если данные уже близки к отсортированным и состоят из длинных естественных серий; средний случай - O(n log n);
- худший случай - O(n log n).

Для моей учебной реализации по памяти можно считать O(n), потому что при слиянии создаются временные списки left_part и right_part. Практические результаты на случайных массивах согласуются с теорией O(n log n).

Коэффициент корреляции между размером массива и временем составил 0.9839, а между размером массива и количеством шагов - 0.9926. Отношение steps / (n log2 n) в среднем равно 2.342, меняясь в пределах примерно от 2.119 до 2.914. Это показывает, что рост числа шагов близок к функции n log n.

8. Выводы

В ходе выполнения семестровой работы был реализован алгоритм TIMSORT на Python.

- создана программа генерации 100 входных файлов размером от 100 до 10 000 элементов;
- реализовано чтение массивов из файлов перед запуском сортировки;
- добавлены два способа измерения: по времени выполнения и по количеству шагов;
- построены графики зависимости времени и шагов от размера входных данных;
- выполнено сравнение практических результатов с теоретической сложностью.

По итогам эксперимента можно сделать вывод, что реализованный алгоритм работает корректно и на случайных массивах демонстрирует рост трудоёмкости, близкий к O(n log n).